

Gas Station Audit



June 16, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	6
Security Model and Trust Assumptions	8
Additional Considerations	9
Review of PR #1 — A PoC to Freeze the Contract	9
Medium Severity	11
M-01 _approveThenExecuteNoReturnWithParams Places Approve Return Data at Calldata Start	11
M-02 Signed Execution Structs Lack Gas Parameter, Allowing Relayer to Influence Execution Outcomes	12
Low Severity	13
L-01 Nested Station/Self Calls Allow Bypassing MAX_BATCH_SIZE	13
L-02 nonceEnd Variable Name Does Not Reflect the Field It Bounds	13
L-03 Zero-Argument burnNonce Consumes the Next Available Nonce Rather Than the Intended One	14
L-04 Constructor Comment States Incorrect EIP-712 Domain Version	14
L-05 approveThenExecute Bypasses USDT Front-Running Mitigation	15
L-06 Session Actions Lack User-Defined ETH Spending Cap	15
L-07 Single Signature Valid for Both Returning and Non-Returning Execution Variants	16
L-08 Missing Calldata Length Validation	16
L-09 Incomplete Batch Session ABI Validation	17
L-10 Inconsistent ethAmount Encoding Between Fallback and Named Functions	17
L-11 Inconsistent Error Handling	18
Notes & Additional Information	18
N-01 Mapping in State Struct Lacks Named Parameters	18
N-02 Missing Security Contact	19
N-03 FloatingPragma	19
N-04 Unnecessary unchecked Blocks	19
N-05 Unnamed Magic Numbers Throughout Assembly Code	20
N-06 IERC1721 Interface Filename and Name Reference EIP-1721 Instead of EIP-1271	20
N-07 Custom Errors in require Statements	20
N-08 Code Quality and Readability Improvements	21
N-09 outputContractBytes Variable Is Set but Never Used	21
N-10 Missing Docstrings	21
N-11 TKGasStation Does Not Expose a burnSessionCounter Forwarder	22

N-12 Unnecessary Code Duplication and Inefficient Memory Management	22
Conclusion	24
Appendix	25
Issue Classification	25

Summary

Type	DeFi	Total Issues	25 (24 resolved)
Timeline	From 11-05-2026 To 22-05-2026	Critical Severity Issues	0 (0 resolved)
Languages:	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	2 (1 resolved)
		Low Severity Issues	11 (11 resolved)
		Notes & Additional Information	12 (12 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [tkhq/gas-station](#) repository at commit [ee3424e](#).

In scope were the following files:

```
src
├── TKGasStation
│   ├── interfaces
│   │   ├── IBatchExecution.sol
│   │   ├── IERC1155Receiver.sol
│   │   ├── IERC1721.sol
│   │   ├── IERC721Receiver.sol
│   │   ├── ITKGasDelegate.sol
│   │   └── ITKGasStation.sol
│   ├── TKGasDelegate.sol
│   └── TKGasStation.sol
```

We also reviewed a proposed [pull request](#) to freeze the contract.

System Overview

TK Gas Station is a metatransaction system that enables a third-party paymaster to pay gas fees on behalf of user EOAs. The system is built on EIP-7702, a Pectra-era mechanism that allows an EOA to point its code to a smart contract address. Once delegated, the EOA gains programmable execution capabilities while its private key remains the root of trust for authorizing operations.

The system consists of two immutable contracts. **TKGasDelegate** is the EIP-7702 delegation target: after a user signs a type-4 authorization transaction, their EOA's code field is set to a 23-byte prefix pointing to **TKGasDelegate**. All state (nonces, session counters) is stored at the user's own address under an ERC-7201 namespaced slot and persists across any subsequent re-delegations. **TKGasStation** is an optional routing contract used by paymasters; it validates that a target EOA is properly delegated before forwarding execution calls, and restricts the forwarded call types to execution-only operations (single execute, approve-then-execute, and batch execute). Paymasters may also call **TKGasDelegate** directly, bypassing **TKGasStation**, provided they perform off-chain delegation validation.

TKGasDelegate supports the following execution modes:

- **Standard execute:** single external call authorised by the EOA's private key. The signed EIP-712 struct commits to a sequential nonce, a deadline, the target address, the ETH value, and a hash of the calldata. The nonce is consumed on execution, preventing replay. The no-return variant discards the inner call's return data for gas efficiency; the return variant bubbles it up.
- **Approve-then-execute:** atomically calls **approve(spender, amount)** on an ERC-20 contract and then performs an arbitrary call, in a single signed operation. Includes a USDT-style fallback path that first resets the allowance to zero before setting the desired amount, to handle tokens that reject non-zero-to-non-zero approvals. The signed struct commits to the ERC-20 address, spender, approve amount, target, ETH value, and calldata.
- **Batch execute:** executes an ordered array of **Call structs** (each containing a target address, ETH value, and calldata) in a single signed operation. Each struct is individually EIP-712 hashed and the array hash is included in the outer signature. The batch size is capped at **MAX_BATCH_SIZE** (20). Execution halts on the first failing call.

- **Session execute (single and batch):** the EOA pre-signs a session that grants a specific sender address the right to call a specific outputContract. Subsequent calls from that sender do not require a new EOA signature; replay protection is handled by a counter rather than a nonce. The signed struct does not commit to calldata or ETH value, meaning the sender controls those freely within the session. A batch variant allows the sender to submit multiple calls in a single transaction under the same session authorisation.
- **Arbitrary session execute (single and batch):** session variants with no target restriction, where the signed struct commits only to the counter, deadline, and sender, leaving the sender free to call any contract with any calldata, ETH value, and — in the batch form — any number of calls up to the batch limit.
- **Nonce and session counter management:** `burnNonce` (signed) lets the EOA invalidate its current nonce without executing a call, blocking any pending signed transaction. `burnNonce()` (self-call only) does the same without a signature. `burnSessionCounter` lets the EOA invalidate a session counter with a signature, revoking all session authorisations tied to that counter.

The contract also exposes a compact calldata fallback function. The first payload byte is always `0x00` (distinguishing it from any named function selector in either contract), and the second byte encodes the operation type in its upper nibble and a return-data flag in its lower nibble. This encoding is designed to reduce calldata size for paymaster submissions.

`TKGasDelegator` additionally implements ERC-1271 signature validation, ERC-721 receiver, and ERC-1155 receiver interfaces to support tokens held at the delegated EOA address.

Both contracts are deployed at deterministic cross-chain addresses using a `create2` factory with hardcoded salts.

The draft PR adds an ownership and freeze mechanism to `TKGasStation`: a new owner can call `setDelegate(address)` to swap or nullify the delegate, halting the system if needed. `TKGasDelegator` gains an immutable `GAS_STATION` field that rejects calls from any other address. The fallback dispatcher is also removed entirely.

Security Model and Trust Assumptions

EOA owner: The EOA owner holds the private key used to authorize EIP-7702 delegation transactions and all EIP-712 metatransactions. The owner can burn nonces and session counters to revoke pending authorizations, either by signing a typed-data burn message or by calling the burn functions directly when `msg.sender` is the EOA itself and equals `tx.origin`. All other security guarantees assume the owner's private key remains uncompromised; key compromise allows re-delegation to a different contract or the signing of new sessions and transactions.

Paymaster: The paymaster constructs and submits transactions, paying gas on behalf of users. It can route calls through `TKGasStation` or interact with `TKGasDelegate` directly. The paymaster cannot deviate from the parameters a user has signed, but it controls which authorized operations are submitted and when. Users that grant sessions must trust the paymaster to operate within the scope of the authorization and to cooperate with counter burns when revocation is needed before the deadline.

TKGasStation: `TKGasStation` enforces delegation validity by inspecting the EOA's code: it requires exactly 23 bytes with the `0xef0100` EIP-7702 prefix followed by the `TK_GAS_DELEGATE` address. The `TK_GAS_DELEGATE` value is immutable and set at construction. The gas station's fallback path additionally limits accepted function selectors to execution-type operations (`0x00`, `0x10`, `0x20`), preventing session-based calls from being submitted through it. This prevents any caller from using the gas station as a relay for arbitrary messages without a user-signed payload.

TKGasDelegate: The delegate stores its state at the user EOA's address. If the user re-delegates to a different contract, that contract can read or modify the storage at the nonce and session counter slots. The README identifies this as an accepted risk because an attacker who can cause re-delegation already has key-level access; the practical impact is bounded by transaction deadlines. The contracts are immutable with no administrator and no upgrade path.

Additional Considerations

Arbitrary session execution (`executeSessionArbitrary`, `executeBatchSessionArbitrary`) grants the authorized sender the ability to call any contract on behalf of the user's EOA without restriction on target or calldata. The scope of trust granted by this mode is significantly broader than standard session execution and requires active counter lifecycle management to revoke before the deadline. The README explicitly marks this as a footgun intended for careful use.

The contracts include no reentrancy guards. The sequential nonce provides implicit protection against reentrancy for standard execution paths, since the nonce is consumed before the external call. Session-based paths do not have an equivalent guard and are intentionally replayable. The system relies on target contracts to guard their own invariants against reentrant calls, consistent with the behavior of a standard EOA.

Review of PR #1 — A PoC to Freeze the Contract

This PR makes three structural changes: it removes the fallback dispatcher from `TKGasDelegate`, introduces ownership and an upgradeable delegate pointer to `TKGasStation`, and binds `TKGasDelegate` to a specific gas station address. Each of these moves in a positive direction, with some minor concerns noted below.

Removing the fallbacks simplifies the codebase considerably. The custom 1-byte selector parsing maintained a parallel packed wire format alongside the standard ABI-encoded entry points, which was the source of several encoding inconsistencies noted in the audit. Its removal reduces the auditable surface area and eliminates that class of issues. The gas cost of standard ABI dispatch is negligible relative to the execution cost of the forwarded calls, so the efficiency tradeoff is minimal.

We note that the first version of the `_executeSessionWithValueNoReturn` function and the first version of `_executeSessionArbitraryWithValue` function are now obsolete and can also be removed.

Binding `TKGasDelegate` to a specific `GAS_STATION` address is a meaningful security improvement. By having the delegate reject calls that do not originate from its associated gas

station, any logic added to `TKGasStation` (access control, rate limiting, allowlists, or other policy enforcement) is guaranteed to apply to every execution path. Without this binding, a relayer could bypass `TKGasStation` entirely and call the delegate directly, circumventing any checks the gas station imposes. Users can now trust that `TKGasStation`'s invariants are always enforced.

One implementation oversight is that the new requirement is not yet applied to the `burnNonce` function.

Making `TKGasStation` ownable with an updatable delegate address provides useful operational flexibility. If a bug is found in `TKGasDelegate` or a new version is needed, the operator can point the station to a new implementation without redeploying `TKGasStation` and without invalidating existing user delegations to the station address. The `_isDelegated` check continues to enforce that only correctly EIP-7702-delegated EOAs can be used as targets, so the station itself remains protected regardless of which delegate is active. Care is needed on the user side: users who have 7702-delegated to the old `TKGasDelegate` address will need to re-delegate to the new one, and there is a window during migration where the two populations of users coexist.

The freeze mechanism (setting the delegate to `address(0)`) conflates two distinct operations and should be decoupled. Currently, pausing the system requires calling `setDelegate(address(0))`, which means resuming requires a subsequent `setDelegate(newAddress)` call. This creates an implicit ordering dependency and makes it harder to distinguish in event logs and monitoring tooling between a pause and a genuine delegate upgrade. A dedicated `pause` / `unpause` function would make the intent explicit, allow the existing delegate address to be preserved during a pause, and enable the two actions to be governed independently, for example, allowing a hot key to pause while requiring a multisig to change the delegate.

Medium Severity

M-01

`_approveThenExecuteNoReturnWithParams` Places Approve Return Data at Calldata Start

The `_approveThenExecuteNoReturnWithParams` function constructs an `IERC20.approve` call in inline assembly and invokes it against the ERC-20 contract. The `call` instruction passes the start of the constructed approve calldata as both the input data pointer and the return data pointer, causing any data returned by the ERC-20 contract to overwrite the function selector and `_spender` argument of that calldata. This is inconsistent with the `memory pointer update` that follows, which advances the free memory pointer by an extra word that appears intended to reserve space for a return value placed beyond the calldata.

The misalignment does not produce observable misbehavior in the current implementation because the approve calldata is not reused after the call returns, and the USDT retry path executes only when the first call reverts, which typically leaves the return buffer unchanged. Nonetheless, the layout is incorrect relative to the evident design intent: the extra word reserved at the memory pointer update is never reachable by the return data as currently written. The code is fragile with respect to future modifications that inspect or reuse the approve calldata or return data after the call.

Consider updating the return data pointer in the approve `call` instruction to point to the word immediately after the constructed calldata, consistent with the memory reservation that follows.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

We've decided to remove the approve-then-execute functionality, which includes this vulnerability.

M-02 Signed Execution Structs Lack Gas Parameter, Allowing Relayer to Influence Execution Outcomes

`TKGasDelegator` verifies execution requests against a signed struct that specifies the target address, ETH value, and calldata, but does not include a minimum gas requirement. The relayer therefore has full discretion over how much gas is supplied to the transaction and, by extension, to each internal call. If any called contract branches on available gas, whether through explicit `gasleft()` checks, or by catching out-of-gas exceptions in a way that does not revert the top-level call, the relayer can alter execution outcomes without the user's knowledge or consent. This is an additional trust assumption placed on the relayer that is not apparent from the signed data alone.

The risk is compounded in batch execution. Because each `Call` is executed sequentially and no per-call gas limit is enforced, the gas available to each successive call depends on how much the preceding calls consumed. On-chain gas consumption can differ from the relayer's estimate if state changes from other transactions in the same block cause the called contracts to take different code paths. A relayer that anticipates or influences those differences has a degree of control over which calls succeed, fail silently, or follow alternative paths within a single signed batch. Alternatively, an external user that front-runs the transaction to change the blockchain state could also influence the batch execution.

Consider including a minimum gas field in the signed execution and session structs, and a per-`Call` minimum gas in the batch variants, enforced by a `gasleft()` check immediately before each external call. This allows users to bound the gas conditions under which their signed intent is valid and ensures the relayer cannot selectively alter the gas available to individual calls.

Update: Acknowledged, not resolved. The team stated:

Signed Execution Structs Lack Gas Parameter, Allowing Relayer to Influence Execution Outcomes - As designed. We want the relayer (aka turnkey) to completely control how much gas is too much for the batch or single execution. This prevents the end user from being able to waste or require more gas than expected. In a batch, having separate gas limits means that it can 'fail open' per batch and be caught then continue, having weird behavior rather than 'succeed or fail'. We want the relayer (TK) to control the gas for executing on individual wallets. If we were to bundle separate unrelated wallets in a single transaction, then you would want separate gas limits.

Low Severity

L-01 Nested Station/Self Calls Allow Bypassing MAX_BATCH_SIZE

`TKGasDelegate` enforces a per-signed-batch limit of `MAX_BATCH_SIZE = 20` by validating the length of the immediate `_calls` array before execution. However, individual calls within a batch are not restricted by target address, so a batch entry can target `TKGasStation` or the delegated EOA directly. Such a nested call carries its own independently-verified signature and nonce, and can in turn execute its own batch of up to `MAX_BATCH_SIZE` entries. In the case of batch sessions, the same signature can be reused as long as the same caller initiates the subsequent batch calls. This nesting can be repeated recursively until gas or call-stack limits are reached, multiplying the total number of operations well beyond the intended per-batch ceiling and complicating any off-chain sponsorship policy that assumes a fixed upper bound on downstream calls per transaction.

Consider introducing a reentrancy guard that prevents re-entering batch execution while a batch is already in progress. Alternatively, consider documenting that `MAX_BATCH_SIZE` applies per signed batch payload and does not bound the total number of operations in a transaction.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

| We've deleted `MAX_BATCH_SIZE` functionality and this finding with it.

L-02 nonceEnd Variable Name Does Not Reflect the Field It Bounds

The `nonceEnd` variable is named as though it marks the end of the nonce region in calldata, but it actually points to the end of the deadline field, which follows the nonce. The misleading name creates confusion when reading the offset arithmetic throughout the `fallback` function.

Consider renaming `nonceEnd` to `deadlineEnd` or `headerEnd` to accurately reflect the offset it represents.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

| *This codepath has been deleted.*

L-03 Zero-Argument `burnNonce` Consumes the Next Available Nonce Rather Than the Intended One

The zero-argument `burnNonce` function is intended to invalidate a pending signed transaction by consuming the next nonce before it can be used. There is an inherent race condition: if the pending transaction is sequenced first, the burn will not have its intended effect. However, it will consume the next nonce, which may silently invalidate an unrelated signed transaction.

Consider requiring callers to specify the target nonce when calling `burnNonce`, and reverting if that nonce has already been consumed. This makes the operation idempotent and eliminates the risk of accidentally burning an unintended nonce.

Update: Resolved in [pull request #34](#). The team stated:

| *We've removed `burnNonce`.*

L-04 Constructor Comment States Incorrect EIP-712 Domain Version

The `TKGasDelegate` constructor comment states that the EIP-712 domain version is `"1"`, but the [version string passed to the EIP-712 constructor](#) is `"1.1"`. The mismatch could mislead a reader into constructing an incorrect domain separator when verifying signatures off-chain.

Consider updating the comment to reflect the actual version string `"1.1"`.

Update: Resolved in [pull request #31](#) at commit [b60ff4d](#). The team stated:

| *Corrected version in comment.*

L-05 `approveThenExecute` Bypasses USDT Front-Running Mitigation

USDT prevents the allowance of a spender from being changed between two non-zero values, requiring callers to first set the allowance to zero before setting a new non-zero value. This restriction was introduced to prevent a known front-running attack in which a spender with an existing non-zero allowance can observe a pending re-approval transaction in the mempool and drain both the original and newly approved amounts.

The [USDT retry path](#) in `_approveThenExecuteNoReturnWithParams` explicitly circumvents this restriction by first setting the allowance to zero and then setting it to the requested amount within a single transaction. While the two `call` instructions execute atomically, the combined effect is that `approveThenExecute` can succeed when the current USDT allowance to the spender is non-zero, which is precisely the scenario USDT's restriction was designed to prevent. If a prior `approveThenExecute` left a residual non-zero USDT allowance to a given spender, a subsequent call for the same token and spender will trigger the retry path and grant the new allowance without requiring the residual to be cleared first. An adversarial spender who monitors the mempool can front-run the transaction to drain the residual allowance before the gas station transaction is mined; after the gas station transaction executes, the spender also holds the newly approved amount and can drain that too if the downstream call does not consume it entirely.

Consider removing the USDT-specific retry path to restore the intended security restriction.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

| `approveThenExecute` functionality has been removed, taking this finding with it.

L-06 Session Actions Lack User-Defined ETH Spending Cap

Session-based execution allows a relay to submit calls on behalf of the user against any target, with any ETH value, up to a signed deadline. The signed session struct commits to the counter and deadline but does not include a maximum total ETH value the relay may spend within the session. While the relay is trusted, the absence of a user-specified cap means that a compromised or misbehaving relay can drain the user's entire ETH balance across multiple session calls without exceeding any on-chain constraint derived from the user's signed intent.

Consider adding a `maxEthValue` field to the session execution typehashes and enforcing it by accumulating the ETH value of each call against this cap before executing. This gives users a signed, on-chain-enforceable bound on the relayer's spending authority for the lifetime of a session.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

| *We've deleted session execution code and removed this vuln with it.*

L-07 Single Signature Valid for Both Returning and Non-Returning Execution Variants

`TKGasDelegat` exposes paired execution functions where the main behavioral difference is whether return data is surfaced to the caller, such as `approveThenExecute` and `approveThenExecuteReturns`. Both variants in each pair construct identical EIP-712 struct hashes, so a signature produced for one variant is valid for the other. The relayer therefore has discretion at submission time over which variant to call, and can choose to suppress return data that the user expected to be present. This is not apparent from the signed data and may affect off-chain systems or integrations that condition on return values being returned by the transaction.

Consider including a variant discriminant in the signed struct to bind each signature to a specific execution path, or consolidating each function pair into a single function that always returns data and allows callers to discard it.

Update: Resolved in [pull request #30](#) at commit [6f2a992](#). The team stated:

| *We've deleted the "returns" variations.*

L-08 Missing Calldata Length Validation

The `TKGasDelegat fallback` function partitions the input calldata into a sequence of fields that can be referenced independently. Whenever the final field is an arbitrary length buffer (to be used as calldata for a subsequent call), its size is determined ([1](#), [2](#), [3](#), [4](#), [5](#), [6](#)) by subtracting `calldatasize` from the start of the buffer.

However, if the provided calldata is too small to contain all the expected fields, this subtraction can negative overflow and create an excessively large value as the final field. In practice, this

will cause an out-of-gas revert when the buffer is copied to the next frame as part of the target call.

For the sake of robustness, consider validating that the provided calldata is large enough to contain all the expected fields.

Update: Resolved in [pull request #30](#). The team stated:

| We've deleted `fallback`.

L-09 Incomplete Batch Session ABI Validation

The section of the `TKGasDelegate` `fallback` function intended to handle batch sessions interprets the end of the calldata as an `ABI-encoded IBatchExecution.Call[]` array.

However, the first encoded word (at position `107`) is not confirmed to be the expected `0x20` offset, making it possible to supply a malformed buffer with an arbitrary offset value. While this field is not used in the rest of the execution flow and cannot affect behavior, the validation is still desirable for consistency, especially since the same check is performed in other parts of the codebase ([1](#), [2](#), [3](#), [4](#), [5](#), [6](#)).

For consistency and to remove fragility, consider validating the offset pointer as well.

Update: Resolved in [pull request #30](#). The team stated:

| We've deleted `fallback`.

L-10 Inconsistent ethAmount Encoding Between Fallback and Named Functions

The `fallback` function encodes `ethAmount` as a packed 10-byte (`uint80`) field in all eight value-bearing execution paths (selectors `0x00`, `0x01`, `0x10`, `0x11`, `0x30`, `0x31`, `0x50`, `0x51`), while the [corresponding named functions](#) overloads declare the parameter as `uint256` (32 bytes).

Any client SDK that infers the calldata encoding from the named function ABI (typed as `uint256`) and applies that encoding when constructing fallback calldata will produce transactions that revert at signature verification. There is no path to silent misexecution: an incorrectly encoded `ethAmount` shifts the arguments slice and causes the signature check to fail. The risk is therefore limited to unexpected reverts and degraded usability.

Consider standardizing the encoding across both the fallback and named functions to eliminate the inconsistency and reduce the risk of client-side encoding errors.

Update: Resolved in [pull request #30](#). The team stated:

| We've deleted `fallback`.

L-11 Inconsistent Error Handling

The `TKGasDelegate` contract reverts with the `ExecutionFailed` error ([1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [9](#)) whenever the user-specified call fails and the function returns the result of that call on success. The corresponding functions that do not return the call result all revert with no error ([1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#)), except for `_executeBatchWithCallsNoReturn`.

For the sake of consistency, consider using the `ExecutionFailed` error whenever it applies.

Update: Resolved in [pull request #39](#).

Notes & Additional Information

N-01 Mapping in `State` Struct Lacks Named Parameters

The `expiredSessionCounters` mapping uses unnamed key and value types. Since [Solidity 0.8.18](#), mappings support named parameters to clarify their semantics.

Consider declaring the mapping as `mapping(bytes16 counter => bool expired)` to make its purpose self-documenting.

Update: Resolved in [pull request #30](#). The team stated:

| These functions have been removed.

N-02 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Resolved in [pull request #32](#).

N-03 Floating Pragma

The codebase uses the floating `^0.8.30` pragma directive.

Consider using fixed pragma directives to clearly identify the Solidity version with which the contract will be compiled and to ensure consistent compilation behavior.

Update: Resolved in [pull request #38](#). The team stated:

| [Pinned to 0.8.30](#).

N-04 Unnecessary `unchecked` Blocks

Since Solidity [version 0.8.22](#), the compiler automatically removes overflow checks from `for` loop increments. This means the explicit `unchecked { ++i; }` blocks are unnecessary and add visual noise.

To improve the overall clarity and maintainability of the codebase, consider removing unnecessary unchecked blocks.

Update: Resolved in [pull request #37](#).

N-05 Unnamed Magic Numbers Throughout Assembly Code

The inline assembly in `TKGasDelegate` uses several literal values whose meanings are not apparent from context. Examples include the [function selectors](#), [bit shifts used to extract sub-word values](#), and the many calldata offset constants throughout all parsing functions.

Consider defining named constants for semantically meaningful literals, especially function selectors, bit-shift widths, and calldata offsets, to improve readability and reduce the risk of transcription errors.

Update: Resolved in [pull request #41](#).

N-06 `IERC1721` Interface Filename and Name Reference EIP-1721 Instead of EIP-1271

The `IERC1721` interface is intended to represent [ERC-1271](#) but its name and filename transpose the digits, referencing a non-existent ERC-1721.

Consider renaming the file and interface to `IERC1271` to match the standard it implements.

Update: Resolved in [pull request #36](#).

N-07 Custom Errors in `require` Statements

Since Solidity [version 0.8.26](#), custom error support has been added to `require` statements. Initially, this feature was only available through the IR pipeline. However, Solidity [version 0.8.27](#) extended support for this feature to the legacy pipeline as well.

Throughout the codebase, multiple instances ([1](#), [2](#), [3](#), [4](#), [5](#), [6](#)) of `if-revert` statements were identified. For conciseness and gas savings, consider replacing them with `require` statements.

Update: Resolved in [pull request #33](#).

N-08 Code Quality and Readability Improvements

The following minor improvements are suggested to improve code clarity and maintainability: - In the first branch of the `fallback` function, `outputContract` is assigned the same value in both branches of `a conditional`. The duplicated assignment adds noise without affecting behavior and makes the code harder to follow. - In the `_executeBatchSessionNoReturn` function, the `length` variable should be assigned at the start of the function and then reused to avoid `a redundant calldata read` and simplify the code. - In the `burnSessionCounter` function, the explicit call to the library's `recoverCalldata` function could be replaced with `_requireSelf` to eliminate duplication and improve readability.

Consider addressing all three points to improve the overall readability of the codebase.

Update: Resolved in [pull request #30](#). The team stated:

| All three points were deleted from the codebase.

N-09 `outputContractBytes` Variable Is Set but Never Used

In the `fallback` function, `outputContractBytes` is assigned from calldata but is never read; only `outputContract` is used in subsequent execution paths.

Consider removing the `outputContractBytes` assignment to reduce confusion.

Update: Resolved in [pull request #30](#). The team stated:

| We've deleted this chunk of code.

N-10 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified, including the `fallback` dispatcher and the session execution variants.

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #40](#).

N-11 TKGasStation Does Not Expose a burnSessionCounter Forwarder

TKGasStation provides `burnNonce` forwarders and a `hashBurnSessionCounter` helper but does not expose a `burnSessionCounter` forwarder, and its packed `fallback` explicitly rejects session-mode payloads. A station-only sponsor wishing to revoke a session counter must therefore sponsor an execution-mode call that makes the delegated EOA call itself, which requires an additional nonce and a separate burn-session signature.

Consider adding a `TKGasStation.burnSessionCounter` forwarding function so station-only sponsorship can revoke sessions without consuming an execution nonce.

Update: Resolved in [pull request #30](#). The team stated:

| *We've deleted sessions in this contract, taking this finding with it.*

N-12 Unnecessary Code Duplication and Inefficient Memory Management

The codebase contains several patterns that add complexity and gas overhead without clear benefit.

- Internal execution functions are systematically duplicated into a zero-value and a non-zero-value variant (e.g. `_executeNoValue` / `_executeWithValue`). Every call site selects between the two with an explicit `if (ethAmount == 0) branch`. The branch itself costs gas, and passing `value = 0` to a single unified call is not meaningfully more expensive than omitting it. The duplication roughly doubles the number of internal execution functions without a measurable saving.
- Some functions have two versions and the difference is only the type of one argument. But this does not always have an advantage. For example: The address-accepting version of `_executeBatchSessionNoReturn` is called only by `executeBatchSession`, which extracts the address from a bytes blob immediately before the call. No public entry point allows a caller to supply the address directly, making the overload unreachable without the bytes encoding step and therefore redundant.

- Throughout the contract, assembly blocks that write calldata to memory for a single call advance the free memory pointer even though the written memory is never read by any subsequent Solidity code. The memory-safe annotation does not require this: it requires that the assembly does not write below the free pointer. Writing above the free pointer without updating 0x40 is legal and sufficient when the data will not be read by later Solidity code. Reusing the same buffer across iterations would reduce memory expansion cost proportionally to the number of calls in the batch, at no correctness risk. For a batch of n calls each with d bytes of calldata, the current implementation allocates $n \times d$ bytes of memory; reusing the buffer requires only d bytes.

Consider adopting a more deliberate approach to memory use and avoid many different, almost identical versions of functions, if gas efficiency is a priority.

Update: Resolved in [pull request #42](#). The team stated:

`_executeWithValueNoReturn` and `_executeNoValueNoReturn` merged into `_executeNoReturn`. This removes a lot of the duplicate code.

Conclusion

The audited codebase implements a gas abstraction system for EIP-7702 delegated EOAs, supporting a range of execution modes from single calls to batched and session-based operations. The code was of high quality overall: inline comments were thorough, the assembly blocks were clearly annotated, and the README provided a helpful high-level description of the system architecture and data encoding.

The most notable findings were two medium-severity issues: the absence of a gas parameter in signed execution structs, which allows a relayer to supply insufficient or manipulated gas and influence execution outcomes without the user's knowledge, and a memory pointer overlap in the approve-then-execute path where the return data buffer shares the same address as the calldata buffer. The remaining findings were code quality observations covering minor encoding inconsistencies, redundant function overloads, and suboptimal memory management patterns in assembly loops, none of which represent immediate security risks but would benefit from attention before a wider production rollout.

The Turnkey team was eager to help throughout the engagement, providing prompt responses and useful context about the system's intended behaviour.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.